PRD UKK: To-Do List Website

Penulis: Andrean Rayhan Wijaya · Skema: Asisten Pemrogram Junior · Versi: 1.0 · Tanggal: 9 November 2026

1. Ringkasan

Project latihan UKK berupa website To-Do List interaktif yang dibangun dengan HTML, CSS, dan JavaScript biasa (tanpa framework). Pengguna dapat menambah task, melihat daftar task, mengubah status, menghapus task, dan memfilter task berdasarkan status.

Tujuan utamanya bukan hanya aplikasi yang jalan, tetapi juga kemampuan menjelaskan dan mempertanggungjawabkan setiap bagian kode, karena asesor menilai pemahaman terhadap proses pembuatan.

Catatan penting: dokumen ini adalah rancangan latihan berdasarkan materi bootcamp. Saat ujian, selalu ikuti soal MUK asli. Bila ada perbedaan, soal MUK yang berlaku.

2. Tujuan dan Non-Tujuan

Tujuan

- Seluruh fitur inti berjalan: tambah, tampil, ubah status, hapus, filter, validasi.
- Kode terstruktur: satu fungsi satu tugas, data berupa array of objects.
- ESLint 0 error dan Console browser bebas error.
- Penulis dapat menjelaskan alur kode dengan kata sendiri dan menunjukkannya langsung di kode.

Non-Tujuan

- Tidak memakai framework (React, Next.js, dan sebagainya), tidak ada backend, dan tidak ada database.
- Tidak memakai login atau akun pengguna.
- Tidak mengejar tampilan yang rumit; fungsi lebih penting daripada desain.
- Fitur bonus tidak dikerjakan sebelum fitur inti selesai dan bebas error.

3. Pemetaan ke Unit Kompetensi

Unit	Kompetensi	Bukti di project
J.620100.004.02	Menggunakan struktur data	Task disimpan sebagai array of objects (id, title, status)
J.620100.009.01	Menggunakan spesifikasi program	Tiap kebutuhan diterjemahkan menjadi fungsi (bagian 6)
J.620100.010.01	Menerapkan perintah eksekusi bahasa pemrograman	Menampilkan task ke halaman, event klik, className untuk badge status
J.620100.011.01	Instalasi software tools	VS Code, Node.js (LTS) + npm, browser dengan DevTools, ESLint, Live Server; versi dapat ditunjukkan
J.620100.012.01	Pengaturan software tools	File eslint.config.js dan pengaturan VS Code
J.620100.016.01	Guidelines dan best practices	Penamaan jelas, format konsisten, const/let, perbandingan ===, komentar seperlunya
J.620100.017.02	Pemrograman terstruktur	Sequence, selection (if/else), dan iteration (forEach)
J.620100.025.02	Debugging	Kemampuan mereproduksi, membaca error, melacak, memperbaiki, dan menguji ulang bug

4. Pengguna dan Skenario

Pengguna: satu orang yang ingin mencatat tugas harian.

Skenario utama:

1. Pengguna mengetik judul task lalu menekan tombol Add.
2. Task muncul di daftar dengan status awal To Do.
3. Pengguna mengubah status task menjadi In Progress atau Done.
4. Pengguna memfilter daftar berdasarkan status.
5. Pengguna menghapus task yang tidak diperlukan.

6. Bila judul kosong, sistem menolak dan menampilkan pesan error di layar.

5. Kebutuhan Fungsional

5.1 Fitur Inti (wajib)

Kode	Fitur	Aturan
F1	Tambah task	Judul diambil dari input; task baru berstatus todo dan mendapat id unik
F2	Tampilkan daftar	Semua task tampil di halaman beserta statusnya
F3	Ubah status	Status dapat diganti antara To Do, In Progress, dan Done
F4	Hapus task	Task yang dipilih hilang dari data dan dari tampilan
F5	Filter status	Pilihan All, To Do, In Progress, Done; hanya task yang sesuai yang tampil
F6	Validasi input	Input kosong ditolak, pesan error tampil di layar, dan Console bebas error
F7	Setup tools	VS Code dan ESLint terkonfigurasi dengan 0 error

5.2 Fitur Bonus (hanya setelah fitur inti selesai dan bebas error)

- Simpan data di localStorage supaya tidak hilang saat halaman dimuat ulang.
- Edit judul task.
- Konfirmasi sebelum menghapus.
- Hitung jumlah task per status.

6. Spesifikasi Program ke Fungsi

Kebutuhan	Fungsi	Tugas
User dapat menambah task baru	addTask(title)	Membuat object task baru dan memasukkannya ke array
Daftar task tampil di halaman	renderTasks(list)	Mengosongkan daftar di halaman lalu menampilkan ulang task dari list

Kebutuhan	Fungsi	Tugas
User dapat mengubah status task	updateStatus(id, status)	Mencari task berdasarkan id lalu mengganti statusnya
User dapat menghapus task	deleteTask(id)	Membuang task dengan id tersebut dari array
Task dapat difilter berdasarkan status	filterTasks(status)	Mengembalikan semua task bila status all, atau hanya yang sesuai
Input kosong tidak diterima	validateTitle(title)	Mengembalikan pesan error bila tidak valid, atau teks kosong bila valid

Prinsip: satu fungsi satu tugas. Setelah data berubah (tambah, ubah, hapus, filter), tampilan harus diperbarui dengan memanggil renderTasks.

7. Struktur Data

Satu task adalah satu object; banyak task disimpan dalam satu array sehingga mudah ditampilkan, difilter, dan diubah.

Field	Tipe	Keterangan
id	number	Pengenal unik task
title	string	Judul atau isi task
status	string	Salah satu dari: todo, progress, done

Operasi array yang dipakai: push (tambah), find (cari untuk diubah), filter (hapus dan filter status), forEach (menampilkan daftar).

8. Aturan Validasi

- Judul dipangkas spasinya (trim) sebelum diperiksa.
- Judul kosong ditolak dengan pesan yang jelas di layar, bukan hanya di Console.
- Judul sangat pendek (misalnya kurang dari 3 karakter) boleh ditolak sebagai aturan tambahan; ikuti soal MUK bila ada ketentuan lain.
- Tidak boleh ada error di Console pada kondisi apa pun, termasuk saat daftar kosong.

9. Tampilan (Garis Besar)

- Judul halaman: To-Do List.
- Satu kolom input dengan placeholder dan tombol Add.
- Area pesan error di dekat input.
- Empat tombol filter: All, To Do, In Progress, Done; tombol aktif diberi tanda.
- Daftar task; tiap baris memuat judul, badge status berwarna (class berbeda per status), kontrol untuk mengubah status, dan tombol hapus.
- Tampilan boleh sederhana; yang dinilai adalah fungsi dan kerapian kode.

10. Kebutuhan Non-Fungsional

- Teknologi: HTML, CSS, JavaScript biasa; berjalan dengan membuka index.html atau Live Server.
- Struktur folder:
  - index.html, style.css, script.js
  - package.json, eslint.config.js
  - .vscode/settings.json
- Kualitas kode: penamaan jelas (misalnya taskList), indentasi konsisten 2 spasi, const dan let (tanpa var), perbandingan dengan ===, komentar seperlunya.
- Event dipasang dengan addEventListener, bukan atribut onclick di HTML, supaya fungsi tidak dianggap tidak terpakai oleh ESLint.
- Aturan ESLint minimal: no-unused-vars (warn), no-undef (error), semi (error), eqeqeq (error).
- Global browser yang dipakai harus dideklarasikan di konfigurasi ESLint. Konfigurasi contoh di materi hanya mendeklarasikan document; bila kode memakai console, localStorage, confirm, atau alert, tambahkan juga, jika tidak ESLint akan melaporkan error no-undef.

11. Urutan Pengerjaan

1. Setup project dan ESLint (folder, npm, eslint.config.js).
2. Bangun UI dengan HTML dan CSS (form, daftar, filter).
3. Siapkan data dan render (array tasks, renderTasks).
4. Fitur CRUD (tambah, ubah status, hapus).
5. Filter dan validasi.
6. Debug dan lint final (ESLint 0 error, uji semua fitur).

Fitur inti dikerjakan sampai bebas error dahulu, baru fitur bonus.

12. Skenario Uji

No	Skenario	Hasil yang diharapkan
1	Tambah task dengan judul valid	Task muncul dengan status To Do
2	Tambah task dengan input kosong	Ditolak, pesan error tampil di layar
3	Tambah task dengan input hanya spasi	Ditolak seperti input kosong
4	Ubah status ke In Progress lalu Done	Badge dan data status berubah
5	Hapus task	Task hilang dari tampilan dan data
6	Filter To Do, In Progress, Done, All	Hanya task sesuai yang tampil; All menampilkan semua
7	Filter saat tidak ada task yang cocok	Daftar kosong tanpa error
8	Hapus semua task	Daftar kosong, tidak ada error di Console
9	Buka Console setelah semua skenario	Tidak ada error
10	Jalankan ESLint	0 error

13. Kriteria Selesai (Definition of Done)

- Fitur F1 sampai F7 berfungsi sesuai bagian 5.1.
- ESLint menunjukkan 0 error dan Console bebas error.
- Kode terstruktur dengan fungsi terpisah sesuai bagian 6.
- Folder dan file project rapi, serta project dapat dijalankan dari nol.
- Penulis dapat menjelaskan: kenapa array of objects, bagian sequence, selection, dan iteration, bagaimana task tampil di halaman, di mana konfigurasi linter, dan bagaimana melacak bug.
- Project sudah dilatih dibangun ulang dari nol dengan batas waktu.

14. Risiko dan Mitigasi

Risiko	Mitigasi
Tidak paham kode yang ditulis	Ketik sendiri tiap bagian dan jelaskan baris per baris; minta penjelasan, bukan hanya kode jadi
ESLint error karena global belum dideklarasikan	Tambahkan global yang dipakai di konfigurasi
Cannot read properties of null	Periksa id elemen dan muat script setelah DOM siap (misalnya dengan defer)
Status tidak berubah	Gunakan === dan pastikan tipe id sama
Task tetap tampil setelah dihapus	Panggil renderTasks setelah data berubah
Waktu ujian habis	Kerjakan fitur inti dahulu, bonus belakangan; simpan progres tiap fitur selesai
Ketentuan ujian berbeda dari latihan	Tanyakan ketentuan teknis (laptop, tools, bahasa) ke panitia LSP dan ikuti soal MUK